Distributed Systems Project: Permissioned Blockchain Inventory System

Cancian Matteo **Del Pio Luogo Marco**

Version 1.0 — November 1, 2026

Abstract

Managing inventory across multi-site enterprise environments presents critical challenges regarding data integrity and accountability. Traditional centralized hierarchical systems introduce single points of failure and rely on trusted administrators who can unilaterally modify audit logs. This project proposes a fault-tolerant, permissioned blockchain-based distributed system for enterprise/government asset tracking. The system guarantees immutable, append-only logging of inbound and outbound inventory transactions while supporting human error remediation through cryptographic compensating operations. Operating over a network with fair-loss links under a synchronous model, the platform leverages Proof of Authority (PoA) and Byzantine Fault Tolerance (BFT) consensus to achieve deterministic block finality, high throughput, and resilience against adversarial node behavior.

Contents

Contents **2**

1 Introduction **3**
1.1 Application Overview and Problem Statement . . . 3
1.2 Proposed Solution: Permissioned Blockchain Architecture . . . 3
1.3 Trade-off Analysis of Blockchain Technology . . . 4
1.4 Consensus Mechanism: Proof of Authority (PoA) . . . 4
1.5 Testing Plan and Project Schedule . . . 5

2 Analysis **7**
2.1 Functional Requirements . . . 7
2.2 Non-Functional Requirements . . . 8

3 Project **9**
3.1 Logical Architecture . . . 9
3.2 Protocols and Algorithms . . . 10
3.3 Physical Architecture and Deployment . . . 12
3.4 Development Plan . . . 12

4 Implementation **13**
4.1 Software Components . . . 13
4.2 Consensus and Communication Flow . . . 13
4.3 Persisted Block Certificate . . . 14
4.4 Client Consistency . . . 14

5 Validation **15**
5.1 Automated Validation . . . 15
5.2 Integration and Load Validation . . . 15
5.3 Fault Injection . . . 15

6 Conclusions **21**
6.1 Summary and Fault-Tolerance Assumptions . . . 21
6.2 Future Improvements . . . 21

A Source Code and Setup Instructions **23**

Chapter 1

Introduction

1.1 Application Overview and Problem Statement

In modern enterprise environments operating across geographically distributed sites (like national or continental logistics hubs), managing inventory with high reliability is a primary operational requirement. Organizations must track every item movement seamlessly, covering two fundamental workflows:

- **Inbound Operations:** Recording newly arrived goods or asset creations into the global inventory system.
- **Outbound Operations:** Documenting the dispatch, sale, or permanent removal of assets from the inventory.

Centralized inventory databases suffer from fundamental vulnerabilities and limitations. A compromised system administrator or external attacker can alter historical logs, modify asset quantities retroactively, or erase operational records without detection. Furthermore, centralized infrastructures represent Single Points of Failure (SPF) vulnerable to hardware faults, network partitioning, and physical disasters.

Conversely, human operators and automated IoT scanners inevitably commit operational errors (misscanning barcodes, miskeying quantities, or logging duplicate items). In an immutable ledger, simply deleting or modifying past entries is forbidden by design. So, a robust distributed inventory system must provide a mechanism for *error remediation* through explicit, auditable compensating transactions without compromising historical integrity.

1.2 Proposed Solution: Permissioned Blockchain Architecture

To address these challenges, we design a distributed inventory management platform backed by a **Permissioned (Consortium) Blockchain**. Unlike public permissionless blockchains (e.g., Bitcoin or Ethereum), where unknown participants compete via energy-intensive Proof of Work (PoW), a permissioned blockchain restricts ledger validation to pre-authorized enterprise nodes.

The system relies on two primary node types:

1. **Validator/Blockchain Nodes:** Enterprise servers deployed at key regional headquarters or trusted data centers. They maintain the full replicated ledger, run distributed consensus protocols, validate incoming transaction payloads, and assemble transactions into sequential blocks.
2. **Client/Operator Nodes:** User-facing client terminals (web frontends or handheld devices) utilized by warehouse staff. Client nodes do not participate in block generation;

instead, they sign transactions locally and submit them to validator nodes via API endpoints.

By storing item movements as cryptographically signed transaction logs linked through cryptographic hashes, the system ensures strict sequential ordering, immutability, and full accountability. In the implementation, $H(B_n)$ is the SHA-256 digest of the canonical block fields (height, prev_block_hash, transactions).

In this project, we omitted the implementation of **strong authentication methods** for the 'operator'. We therefore assume that transactions are authentically signed by the operators, making it impossible for a Byzantine node to alter them.

1.3 Trade-off Analysis of Blockchain Technology

Applying blockchain technology to enterprise asset management introduces distinct advantages alongside technical trade-offs, summarized below.

Advantages

- **Immutability and Anti-Tampering:** Once a block is committed through consensus, retroactively modifying a transaction requires re-executing consensus across the majority of validator nodes, making fraud virtually impossible.

- **Complete Historical Traceability:** The chain provides an unbroken audit log for every asset from initial registration (inbound) to final disposal (outbound), eliminating operational disputes between warehouse sites.

- **High Infrastructure Resilience:** Replicating the state across independent regional validator nodes guarantees high availability; the failure or destruction of a single site does not cause data loss or system downtime.

Disadvantages and Mitigations

- **Error Remediation Complexity:** Because records cannot be erased, mistake correction requires introducing new explicit compensating transactions (e.g., explicit reversals) linked to the original erroneous transaction ID.

- **Latency vs Throughput:** Distributed consensus introduces message overhead compared to traditional relational databases. Using Proof of Authority (PoA) minimizes this performance cost.

1.4 Consensus Mechanism: Proof of Authority (PoA)

Because all validator nodes represent known corporate entities, consensus is achieved via **Proof of Authority (PoA)** and leader-based Byzantine Fault Tolerance (BFT) algorithms (e.g., PBFT).

In PoA, round-robin or stake-weighted validator rotations designate a block proposer for each time interval. The proposer aggregates pending transactions, signs the proposed block, and broadcasts it to peer validators. Peer nodes verify the proposer's digital signature and the validity of all contained transactions before appending the block to their local copy of the ledger.

1.5 Testing Plan and Project Schedule

Testing Strategy

- **Unit Testing:** Verification of modules and functions independently through specific test suites.
- **Integration Testing:** Testing client-to-validator communication, transaction propagation, and state machine transitions.
- **System & Fault Tolerance Testing:** Simulating node crashes, network packet loss, message reordering, and malicious/Byzantine transaction proposals.

Project Schedule

The project design and implementation follow a four-stage timeline:

1. **Phase 1 (Requirements & Architecture):** Defining system specifications, network models, and UML sequence diagrams.
2. **Phase 2 (Core Development):** Implementing the permissioned blockchain node, networking functionlities, and basic consensus mechanism.
3. **Phase 3 (Client Layer):** Developing client APIs, operator UI, and compensating transaction handling.
4. **Phase 4 (Validation & Evaluation):** Conducting benchmark simulations, network failure injection, and report finalization.

Chapter 2

Analysis

In this chapter, we outline the functional and non-functional requirements for the proposed blockchain inventory system.

2.1 Functional Requirements

FR-1: Inbound Items Registration

- **Description:** Allows an authorized operator to register new assets into the global inventory.
- **Input:** Asset details (ID, description, batch number, quantity, location), operator signature.
- **Output:** Transaction hash and confirmation of block inclusion.
- **Effect:** Increases global asset count; creates initial cryptographic provenance record on the ledger.

FR-2: Outbound Items Tracking

- **Description:** Registers the dispatch or removal of existing assets from stock.
- **Input:** Asset ID, quantity removed, destination details, operator signature.
- **Output:** Transaction receipt or error notification (if insufficient stock exists).
- **Effect:** Appends outbound movement record to the chain.

FR-3: Error Remediation (Compensating Transactions)

- **Description:** Correct errors without overwriting historical records.
- **Input:** Target erroneous Transaction ID, correction type (reversal, adjustment), reason string, operator signature.
- **Output:** Remediation transaction receipt linked to the original transaction ID.
- **Effect:** Updates current inventory state while maintaining a complete audit trail of the original error and its correction.

FR-4: State Querying and Historical Checking

- **Description:** Enables operators to inspect current stock levels or trace an asset's complete lifecycle.
- **Input:** Asset ID, name, time interval query parameters.
- **Output:** Formatted list of chronological historical blocks and state transitions associated with the asset.

2.2 Non-Functional Requirements

NFR-1: Security & Cryptographic Integrity The demo uses plain HTTP for client-to-Python and Python-to-Elixir communication. HTTPS and authenticated inter-site transport are future hardening measures. Transactions use Ed25519 signatures and blocks are cryptographically linked using SHA-256 hash digests.

NFR-2: Availability & Fault Tolerance For Byzantine fault tolerance, the validator configuration must satisfy $n \geq 3f + 1$. This project configures $N = 4$ validators and assumes at most $f = 1$ faulty validator. A block requires a quorum of $2f + 1 = 3$ distinct validator signatures; progress therefore requires at least $N/2 + 1 = 3$ healthy validators and an available proposer for the current round.

NFR-3: Consistency & Finality The distributed ledger must deliver deterministic block finality (zero fork probability once a block is confirmed by a majority of validators), ensuring immediate consistency across all warehouse locations.

NFR-4: Performance & Throughput Transactions are collected in blocks of at most 200 entries. Python triggers a proposal at the configured mempool threshold or after the configured idle interval while transactions are pending. The Elixir round timeout is a consensus timeout, not a block-creation interval.

NFR-5: Auditability & Transparency The full history of all operations must be verifiable by any validator node (and thus by operators) without relying on a centralized central authority.

Chapter 3

Project

This chapter describes the system models, logical architecture, protocols, and deployment structure.

3.1 Logical Architecture

To formally reason about correctness, fault tolerance, and consensus guarantees, we define our system parameters across three dimensions:

- **Network Model (Fair-Loss):** Communication links between nodes are assumed to be **fair-loss**. Messages may be lost, duplicated, or reordered during transmission. However, if a sender retries transmitting a message infinitely often, it will eventually be delivered to the destination.
- **Node Model (Adversarial / Byzantine):** Validator nodes can exhibit arbitrary Byzantine behavior. A compromised or faulty node may crash, delay messages, drop valid transactions, forge signatures, or send conflicting messages to different peers.
- **Timing Model (Synchronous):** Message propagation latency across the network is bounded by a known constant Δ. Nodes operate with bounded execution speed drift, allowing the system to utilize timeout-based round rotas for consensus leader selection.

System structure

The system is structured into two logical layers (Figure 3.2):

1. **Presentation / Client Layer:** Consists of client software executed in business or personal devices operated by warehouse workers ($\text{Operator}_1, \text{Operator}_2, \dots$). It manages local key pairs, captures user input, constructs transaction structures, signs payloads, and submits them to accessible validator nodes.
2. **Blockchain / Backend Layer:** The decentralized core comprising interconnected validator servers ($\text{Server}_X, \text{Server}_Y, \text{Server}_Z$). Each server houses:
   - *Mempool Manager:* Holds pending, unconfirmed transactions.
   - *Consensus Engine:* Executes the Proof of Authority / BFT protocol.
   - *Cached State Updater:* Applies verified blocks to update local inventory state tables.
   - *Ledger Storage:* Append-only disk storage containing linked blocks and indexed databases for fast state queries.

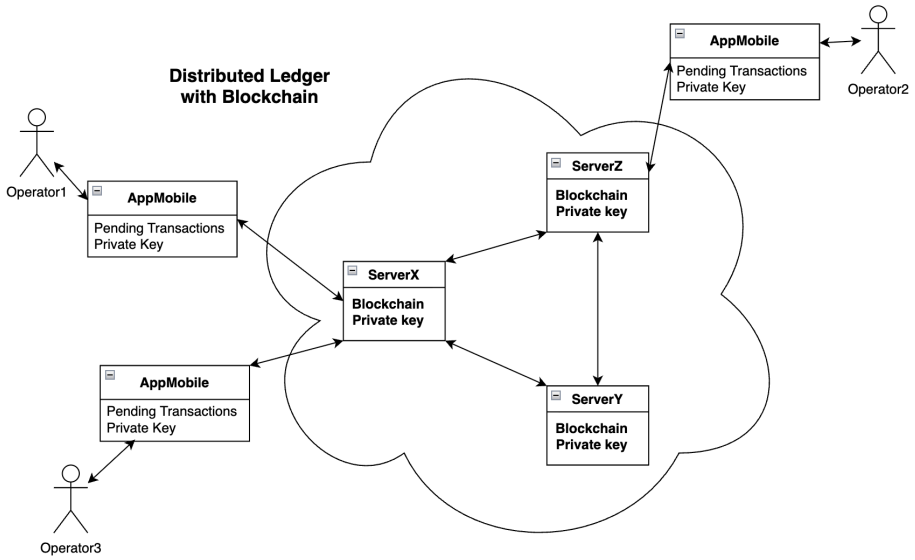


Figure 3.1: Logical Architecture and Component Interactions of the Distributed Ledger System.

3.2 Protocols and Algorithms

Transaction Lifecycle and Block Proposal

The step-by-step operational flow for inventory updates is illustrated in Figure 3.3:

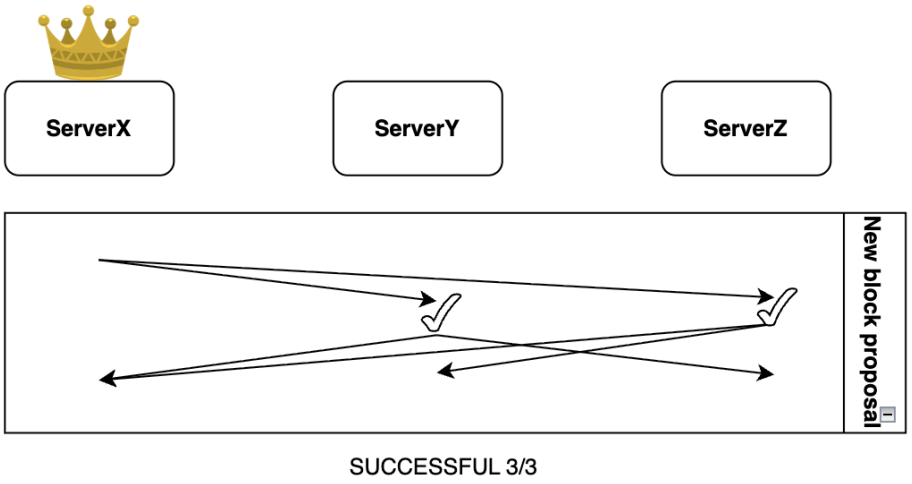
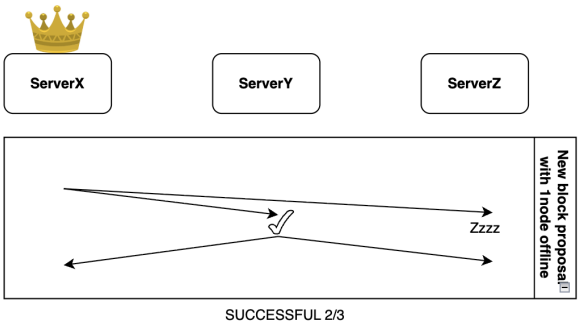


Figure 3.2: Logical Architecture and Component Interactions of the Distributed Ledger System.

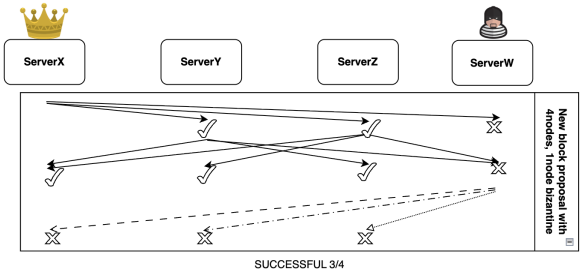
1. **Submission:** The client sends an Ed25519-signed transaction containing `item_id`, signed `quantity`, `location`, `client_ID`, `client_public_key`, `trans_id`, `timestamp`, and `item_catalog` to one Python validator. Positive quantities add stock; negative quantities remove stock.

2. **Broadcast & Pooling:** The receiving Python validator validates and stores the transaction, then forwards it through its co-located Elixir node. A validator receiving it from Elixir stores it without forwarding it again.

3. **Block Assembly:** The deterministic proposer collects pending transactions into a block candidate containing height, round, previous block hash, transactions, and the block hash, then signs the proposer payload.



(a) Sequence Diagram: Transaction Submission & Validation.



(b) Sequence Diagram: Consensus Block Propagation.

Figure 3.3: UML Sequence Diagrams outlining component interactions during operation execution.

4. **Consensus Vote:** The proposer broadcasts PRE-PREPARE. Peers verify the proposal and participate in PREPARE and COMMIT voting. Prepare signatures are protocol-internal; three distinct valid COMMIT signatures form the final certificate.

5. **Commit & State Update:** Each Python validator writes the block only after the COMMIT quorum, updates its inventory cache, and marks included transactions as `finalized`. The client observes this through transaction-status polling and blockchain synchronization; there is no separate leader-to-client broadcast.

Error Remediation Protocol

When an operational mistake occurs (e.g., logging 100 items inbound instead of 10), the operator initiates a compensating flow:

1. Operator selects a 'correction' button inside the frontend dashboard and then selects the transaction T_{err} to be corrected.

2. The software generates an automatic compensation transaction T_{remedy} to cancel T_{err} while the operator creates the new correct transaction $T_{\text{corrected}}$.

3. Once the new transaction is complete, it is signed and sent to the validator node of the warehouse.

Node Recovery and Network Synchronization

When a server crashes and recovers, it must synchronize its mempool and blockchain state securely without relying on naive majority votes, which could be manipulated by a Byzantine node.

- **Mempool Synchronization:** The recovering node requests pending transactions from any active peer. Because every transaction includes the unforgeable client signature (Sig_{Op}), the node independently verifies cryptographic validity without requiring network consensus or trust in the sender.

- **Ledger Synchronization:** The node queries peers for missing blocks. It accepts historical blocks as canonical only if they contain the required $2f + 1$ cryptographic signatures from distinct validators, decoupling the recovery process from the trust level of any single node (including the current leader).

3.3 Physical Architecture and Deployment

The physical deployment model relies on containerized microservices (Docker) across distributed sites:

- Each regional warehouse hosts a local validator container divided in two components: The blockchain module and the networking module.
- Client devices connect via HTTP REST endpoints exposed by the Python API. HTTPS is not enabled in the demo.
- Peer-to-peer communication among validator containers uses the internal Erlang-term TCP listener on port 4000; authenticated encryption and VPN integration are future deployment work.

3.4 Development Plan

To manage implementation risk, development is divided into three incremental tiers:

Tier 1: Blockchain, Networking & Transaction Handling

- In-memory append-only blockchain storage module and Mempool manager.
- Single-node API server with basic CLI client for inbound/outbound logging.
- Cryptographic signature generation and SHA-256 block linking.

Tier 2: Distributed Consensus & Leader Election

- Multi-node network execution with PoA and consensus implementation (with anti-byzantine behaviour).
- Disk persistence engine for state recovery.
- Compensating transaction protocol for error remediation.

Tier 3: Advanced Features & Fault Injection

- Web-based dashboard interface for warehouse operators.
- Simulated Byzantine fault injection framework for robustness testing under network partition and node crash conditions.

Chapter 4

Implementation

4.1 Software Components

The Flutter client uses `go_router` and Provider for routing and state management. It signs transactions locally and calls the Python API. Each Python validator uses FastAPI, SQLAlchemy, and SQLite. The chain, pending transactions, forwarding queue, and validator vote records are persisted in SQLite; the inventory cache is rebuildable from committed blocks. Each Elixir node contains the HTTP bridge, the Erlang-term cluster listener, the router, and the consensus engine.

4.2 Consensus and Communication Flow

The system has four validators. They use a fixed order to select the proposer for each height and round. A block can contain at most 200 transactions. Python asks Elixir to start a proposal when the pending pool reaches 200 transactions, or when transactions have been pending for 10 minutes.

The consensus steps are:

1. Python asks the local Elixir node to start a proposal when a proposal trigger fires. Elixir starts it only if this node is the proposer for the current round. The proposer then asks its Python service to build and sign the block.
2. The proposer sends a `PRE-PREPARE` message to the other validators.
3. Validators check the proposer, block, and signature. They then send signed `PREPARE` votes.
4. After a quorum of three valid `PREPARE` votes, validators enter the `COMMIT` phase and send signed `COMMIT` votes.
5. After three valid `COMMIT` votes, each validator stores the block in its local database. The block keeps the COMMIT certificate.

If a round does not finish within 8 seconds, validators send signed `ROUND_CHANGE` messages. A new round starts after a quorum of three valid messages. The next proposer is then selected by the fixed order.

Communication Between Components

- **Client to Python:** The client sends transactions to a Python REST API over HTTP. It signs each transaction before sending it. Python checks the request, stores the transaction, and forwards it to the other validators. The client can later ask Python for the transaction status.

- **Python to Elixir:** The two components on a validator use an internal HTTP interface. Python asks Elixir to start a proposal when a trigger is reached. Elixir asks Python to build blocks, create or check signatures, and save committed blocks.
- **Between validators:** Elixir nodes exchange consensus messages over TCP. Messages use Erlang's external term format. Python HTTP is not used for validator-to-validator consensus traffic.

Byzantine Test Modes

The Byzantine mode is off by default. It is activated on a node through the `/api/v1/simbehaviour/bizantine` endpoint. The `BIZANTINE_SCENARIO` setting selects the fault. Transactions are still accepted and forwarded; the fault changes only consensus messages.

- `invalid_signature`: The faulty node sends votes or proposals with invalid signatures. Other validators reject them. The remaining three validators can still reach a quorum.
- `split_brain`: If the faulty node is proposer, it sends the COMMIT proposal to only two peers and does not send its own COMMIT vote. The peers share valid COMMIT votes. The three honest validators can still form a quorum.
- `equivocation`: If the faulty node is proposer, it sends two different, validly signed proposals to separate groups. A validator that sees both reports proof of the conflict. If it has not started COMMIT, it stops voting for that block and asks for a round change. If it has already started COMMIT, it may finish. Afterward, validators that committed send the block and its COMMIT certificate only to the reporting validator, so it can catch up.
- `leader_offline`: The faulty proposer stops sending the COMMIT proposal and its `ROUND_CHANGE` vote. Honest validators time out, exchange their own round-change votes, and select a new proposer after reaching a quorum.

4.3 Persisted Block Certificate

The public block representation contains the proposer identifier and signature plus the COMMIT signatures of the validators:

```
proposer_id, proposer_signature, validator_signatures.commit
```

PREPARE signatures are used to authorize the COMMIT phase but are not persisted or displayed to clients.

4.4 Client Consistency

The client keeps an in-memory cache for transactions submitted by the current user. Before applying pending inventory overlays, it reconciles this cache with the authoritative `/user_transactions` response. Finalized or rejected transactions are removed from the cache, preventing a committed quantity from being applied twice.

Chapter 5

Validation

5.1 Automated Validation

The repository includes Flutter widget tests, Elixir ExUnit tests, and Python service tests. The validation workflow also runs Dart analysis, Elixir formatting checks, Python compilation, and Docker health checks.

5.2 Integration and Load Validation

The load-test script submits signed transactions through an Elixir router and waits for committed blocks. Since a block contains at most 200 transactions, larger runs require multiple blocks. The API exposes chain history, blockchain synchronization, pending transactions, transaction status, and forwarding-monitor endpoints for operational verification.

5.3 Fault Injection

The deployment supports test modes for invalid signatures, split-brain behaviour, equivocation, and an unavailable leader. These modes are intended for controlled demonstrations. With four validators, the normal quorum is three; safety assumes at most one Byzantine validator, while availability also depends on the selected proposer and network connectivity.

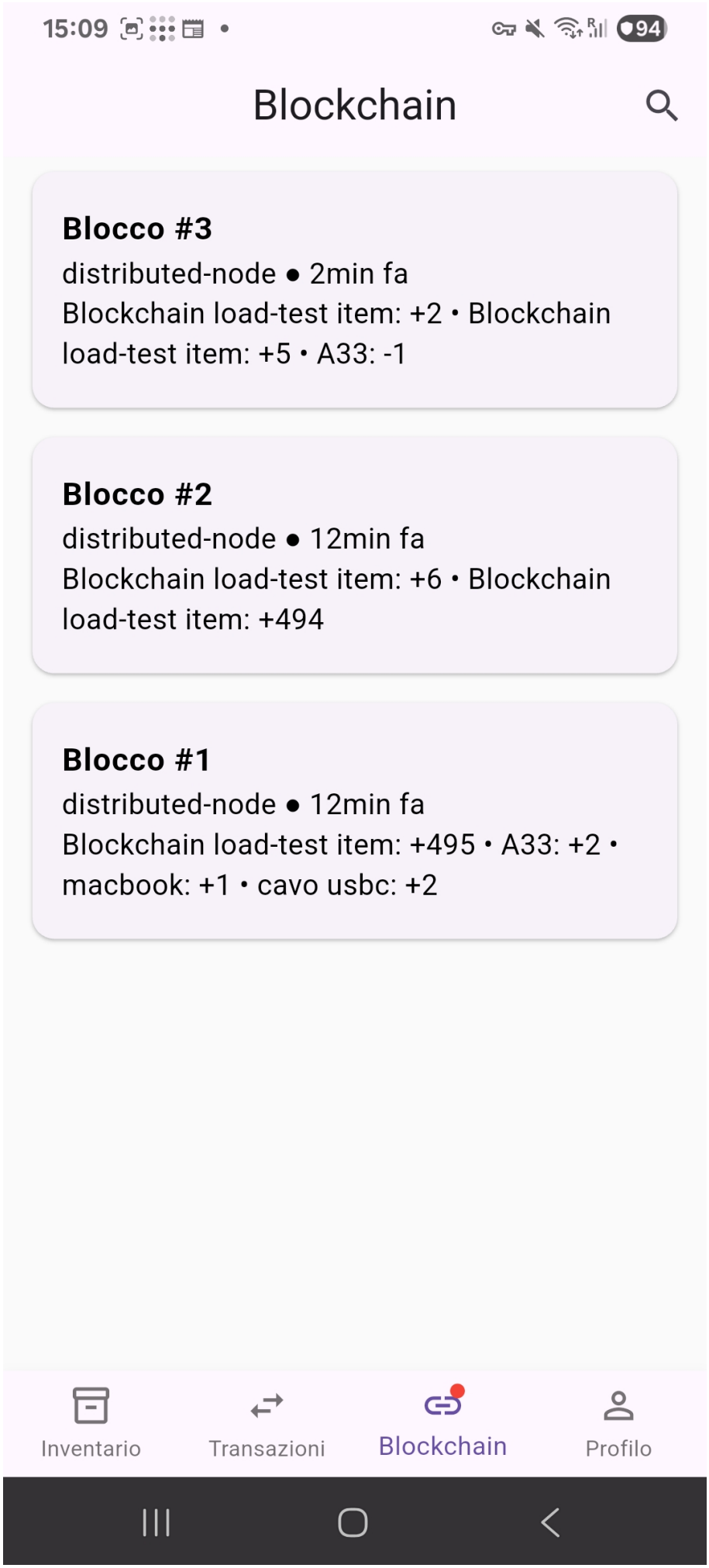


Figure 5.1: Blockchain network visualization

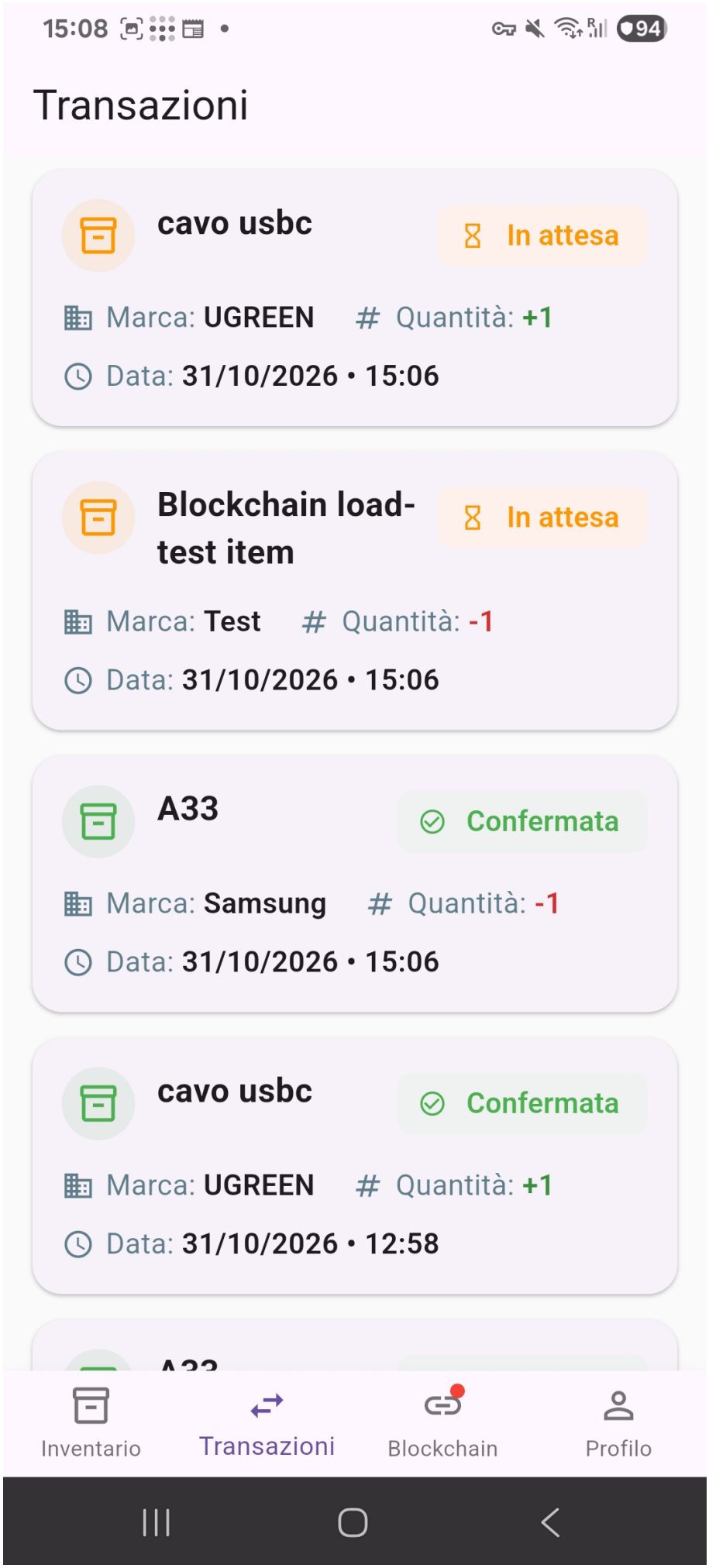


Figure 5.2: Transaction flow in the blockchain system

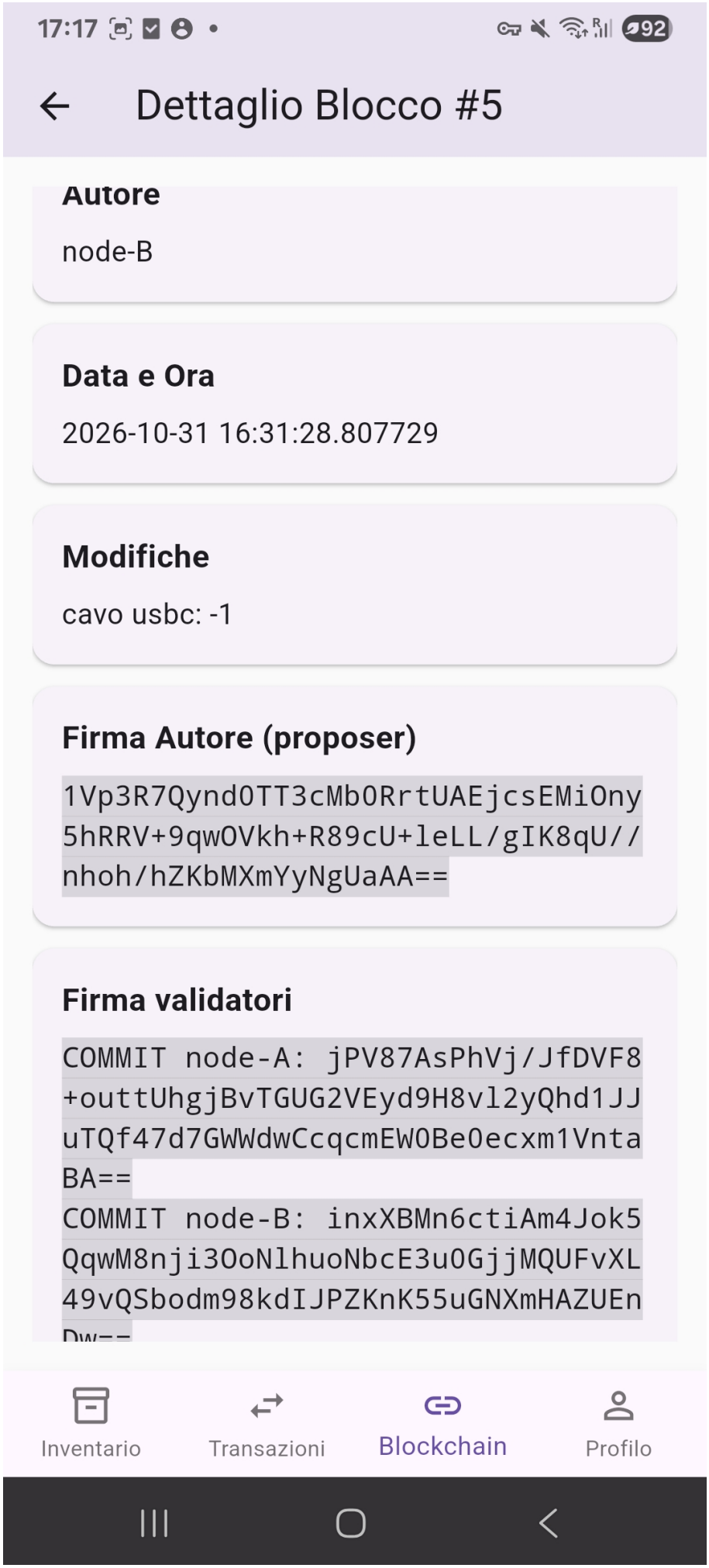


Figure 5.3: Block details and structure

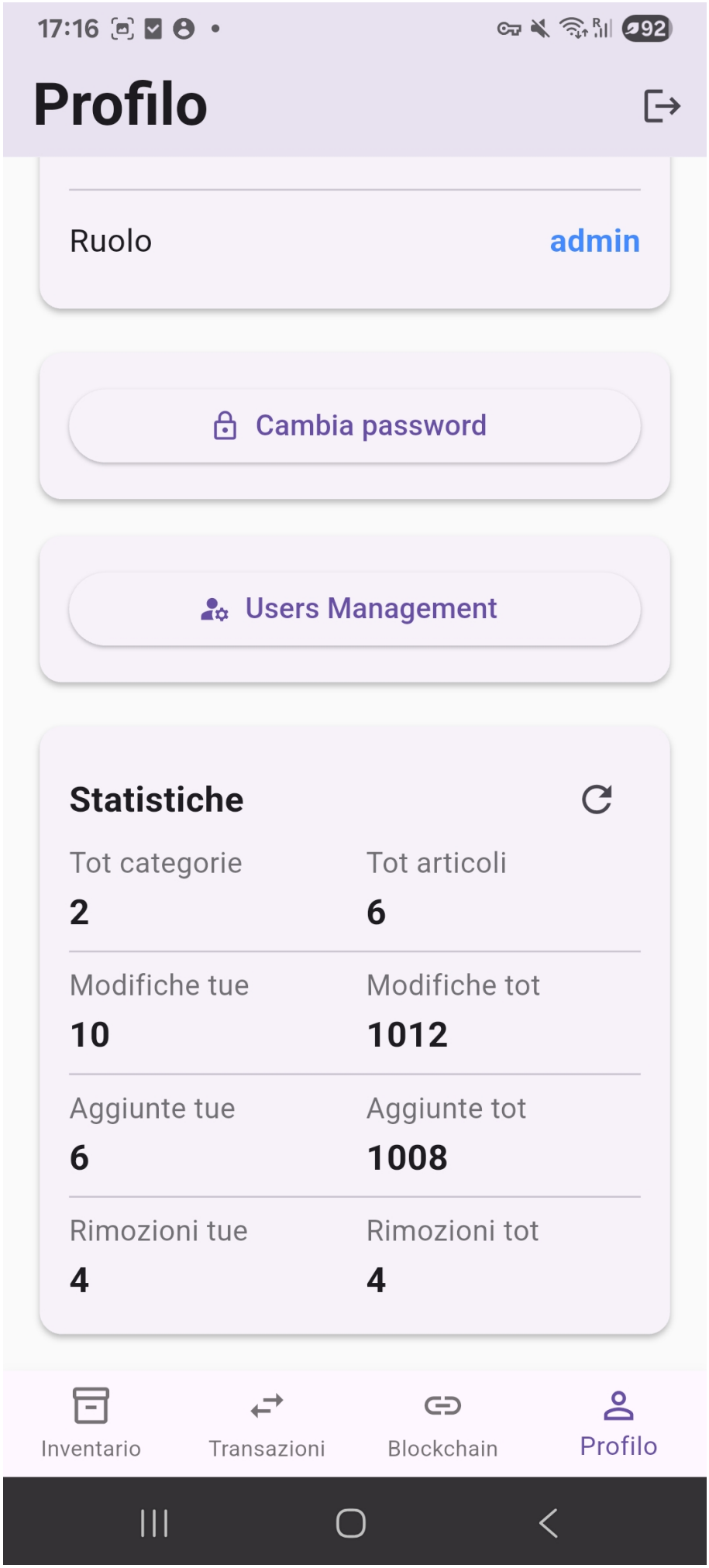


Figure 5.4: Operator profile/dashboard

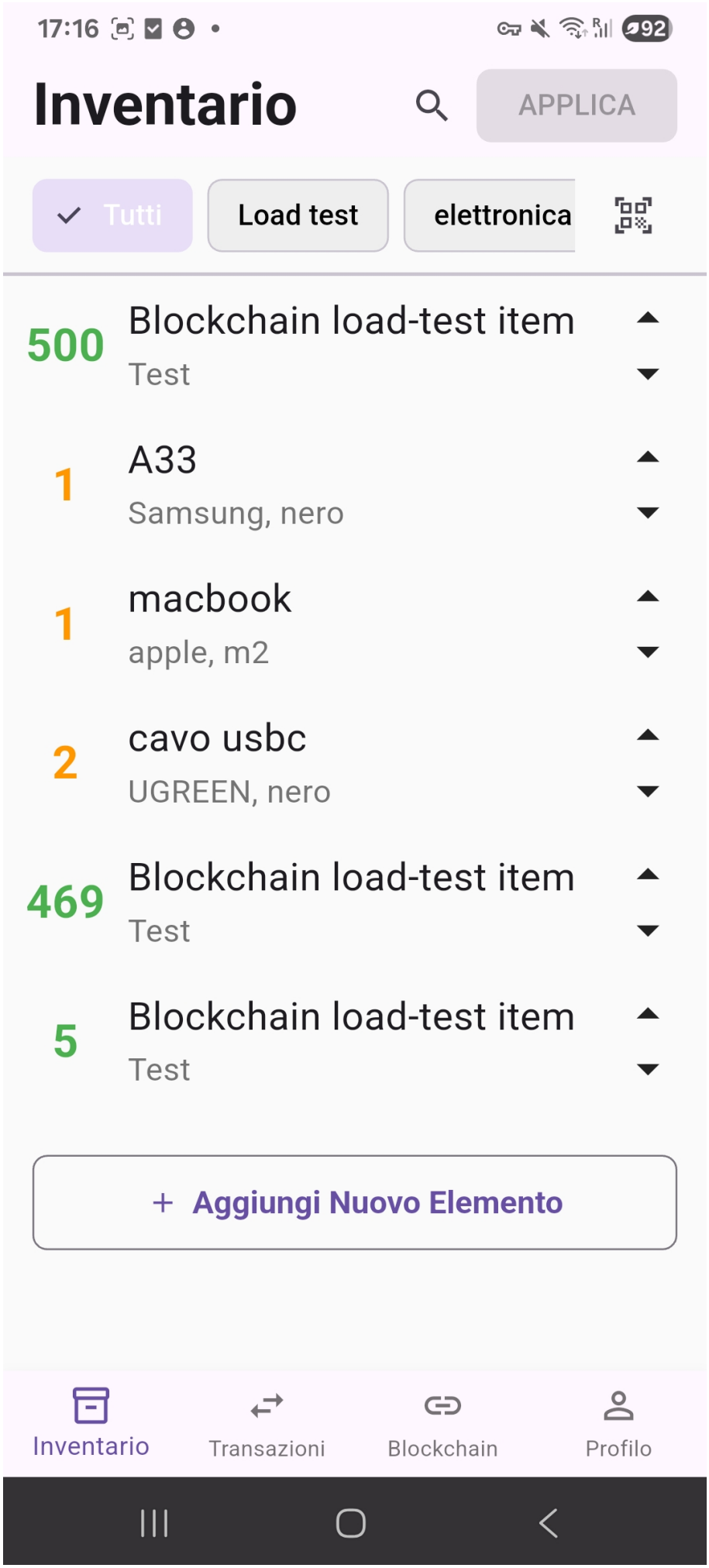


Figure 5.5: Main screen with items and quantities

Chapter 6

Conclusions

6.1 Summary and Fault-Tolerance Assumptions

The design uses four permissioned validator nodes and accepts a block only when it has signatures from at least three distinct validators. Thus, the system requires at least $N/2 + 1 = 3$ healthy validators to make progress. With $N = 4$, this operating assumption permits at most one validator to be unavailable or Byzantine at a time. The proposer may count as one of the three validators, provided it independently validates the proposal and signs the same block hash.

The one-vote-per-height rule is central to the current safety model: an honest validator signs at most one block hash for a given block height. Any two three-validator quorums in a four-validator system intersect in at least two validators. With at most one Byzantine validator, their intersection contains at least one honest validator, which cannot have signed conflicting blocks at that height. Therefore two conflicting blocks cannot both obtain certificates under the stated fault assumption.

This quorum argument concerns safety, not availability by itself. The current implementation selects a proposer deterministically for each height and only accepts proposals from that proposer. If that proposer is the unavailable or faulty validator and does not produce a valid proposal, the remaining three healthy validators do not currently take over automatically; pending transactions can remain uncommitted. The implementation also rejects a validator's second vote for a different block at the same height.

6.2 Future Improvements

A future consensus improvement could support repeated voting at one block height across multiple rounds, allowing a later proposer to try after an earlier proposer is silent or faulty. This must not simply remove the one-vote-per-height restriction: validators need a formally specified round-change and locking protocol, including signed evidence of prior votes and rules requiring later rounds to preserve any block on which a quorum has already locked. Without these safeguards, allowing validators to vote again could permit conflicting certificates and forks. The protocol should be specified and tested under the same four-node, at-most-one-fault assumption before this capability is added.

Other future work includes testing proposer timeouts and recovery, validating certificate propagation under message loss and reordering, and binding operator public keys to authenticated BIS accounts.

Additional improvements include timestamp-age validation, an optional nonce/replay cache, persistent pending-cache storage, forwarding metrics and alerts, HTTPS, and authenticated inter-site transport.

Appendix A

Source Code and Setup Instructions

The local deployment requires Docker Compose, Python 3, and OpenSSL. From the repository root:

```
python3 setup_env.py
docker compose up -d --build
```

The Python APIs are published locally on ports 8001–8004 for nodes A–D. Elixir gossip uses internal port 4000 and its Python bridge uses internal port 6000. The complete endpoint map is maintained in `PORTS.md`; protocol details are maintained in `specifiche_tecniche.md`.